5.3. Les exceptions

- éviter l'arrêt du programme en cas d'erreur prévisible
- prévoir les cas potentiels d'erreurs
- associer la définition d'une exception à chaque cas

DECLARE { CONTINUE | EXIT }
HANDLER FOR
{ *valeur erreur* | NOT FOUND | SQLEXCEPTION }
***Instructions MySQL*;**

CONTINUE : directive qui force à poursuivre l'exécution du programme
EXIT : directive qui fait sortir l'exécution du bloc courant (entre BEGIN et END)
SQLWARNING : permet de couvrir toutes les erreurs d'état identifiées par 1xxx
1007 : ne peut créer la base; elle existe déjà
1008 : ne peut effacer la base; elle n'existe pas
1009 : ne peut effacer la base
1048 : le champ ne peut être vide (null)
1050 : la table existe déjà
1051 : table inconnue
1054 : champ inconnu dans *table*
1172 : le résultat contient plus d'un enregistrement
Et bien d'autres erreurs : *http://dev.mysql.com/doc/refman/5.0/fr/error-handling.html*
NOT FOUND : permet de couvrir toutes les erreurs d'état identifiées par 2xxx
http://dev.mysql.com/doc/refman/5.0/fr/error-handling.html

SQLEXCEPTION : permet de couvrir toutes les autres erreurs

Instructions MySQL : SELECT … ; CALL …

Exemple n°1 – existence du numéro de client

```
CREATE PROCEDURE sp_numcli_existe (IN num INT)
BEGIN
    DECLARE flagNotFound BOOLEAN DEFAULT false;
    DECLARE v_nom VARCHAR(20);
    BEGIN
        DECLARE EXIT HANDLER FOR NOT FOUND
                SET flagNotFound := true;
        SELECT nom_cli INTO v_nom FROM client
        WHERE num_cli = num;
        SELECT CONCAT('Le client ',num, ' – ', v_nom, ' existe.') AS resultat;
    END;
    IF flagNotFound THEN
        SELECT CONCAT('Le client ',num, 'n''existe pas !') AS resultat ;
    END IF ;
END ;
```

Exemple n°2 : existence du nom dans la table client

*a) avec la directive **EXIT***

```
CREATE PROCEDURE  sp_nomcli_existe_avecEXIT (IN nom VARCHAR(20))
BEGIN
    DECLARE   flagNotFound   BOOLEAN    DEFAULT false;
    DECLARE   flagPlusDun    BOOLEAN    DEFAULT false;
    DECLARE   v_num      INT;
    BEGIN   -- début de bloc
        DECLARE   EXIT   HANDLER   FOR   NOT FOUND
                  SET    flagNotFound := true;
        DECLARE   EXIT   HANDLER   FOR   1172
                  SET    flagPlusDun := true;
        SELECT    num_cli  INTO  v_num      FROM  client
        WHERE     nom_cli = nom;
        SELECT    CONCAT('Le seul client ',nom, ' a le numéro ', v_num)   AS      resultat;
    END;    -- fin de bloc
    IF    flagNotFound    THEN
        SELECT    CONCAT('Aucun client ne porte le nom ',nom)    AS    resultat ;
    ELSEIF   flagPlusDun   THEN
        SELECT    CONCAT('Plusieurs clients portent le nom ',nom)   AS   resultat ;
    END  IF ;
END ;
```

*b) avec la directive **CONTINUE***

```
CREATE PROCEDURE  sp_nomcli_existe_avecCONTINUE (IN nom VARCHAR(20))
BEGIN
    DECLARE    flagNotFound     BOOLEAN    DEFAULT false;
    DECLARE    flagPlusDun      BOOLEAN    DEFAULT false;
    DECLARE    v_num        INT;
    DECLARE    CONTINUE    HANDLER       FOR    NOT FOUND
               SET         flagNotFound := true;
    DECLARE    CONTINUE    HANDLER       FOR    1172
               SET         flagPlusDun := true;
    SELECT     no_at  INTO  v_num    FROM    attribut
    WHERE      nom_at = nom;
    IF       flagNotFound    THEN
             SELECT    CONCAT('Aucun attribut ne porte le nom ',nom)       AS      resultat ;
    ELSEIF   flagPlusDun     THEN
             SELECT    CONCAT('Plusieurs ''attributs portent le nom ',nom)   AS      resultat ;
    ELSE
             SELECT    CONCAT('Le seul attribut ',nom, ' a le numéro ', v_num)    AS    resultat;
    END IF;
END ;
```

Exercices

E531 : corriger la procédure **sp_fibonacci (↓ nombre : entier)** *de l'exercice E504* **Durée : 10 min.**

Rappel du problème

Par définition, une table temporaire cesse d'exister à partir du moment où elle est supprimée explicitement avec un DROP ou bien implicitement à la fin de la session de connexion. L'appel de la procédure lors de l'ouverture de la session va générer l'erreur suivante :

call sp_fibonacci(15) *Error Code: 1051. Unknown table 'TMP_FIBO'*

Ce problème est contourné avec la **gestion d'une exception à écrire**.

E532 : procédure **sp_liste_abonne_annuel ( )** **Durée : 20 min.**

En utilisant la procédure ***sp_abonne_annuel*** développée dans l'exercice ***E521***, écrire la procédure qui liste dans une table temporaire TMP_ABOAN (numéro, identité), tous les abonnés considérés comme des abonnées à l'année.

Exemple

```
CALL sp_liste_abonne_annuel( ) ;
SELECT * FROM TMP_ABOAN ;
```

numéro	identité
1	DUPOND Paul
2	DITRA Fernando

E533 : procédure **sp_maxaffluence (↓ nivid : entier ; ↑ resultat : chaine(20))** **Durée : 30 min.**

A partir de la vue ***E303***, écrire la procédure qui retourne dans ***resultat*** la date (jj/mm) et la valeur de la plus grande affluence du numéro de niveau ***nivid*** passé en paramètre. Il est demandé de traiter le cas où le numéro de niveau n'existe pas.

Exemples

```
CALL sp_maxaffluence(1,@resultat);
SELECT @resultat;
```

@resultat
29/10 : 375

```
CALL sp_maxaffluence(4,@resultat);
```

erreur
Le niveau 4 n'existe pas !

```
SELECT @resultat;
```

@resultat
NULL

E534 : procédure **sp_niveau_maxaffluence ( )** **Durée : 30 min.**

En utilisant la procédure stockée **sp_*maxaffluence***, écrire la procédure qui liste dans une table temporaire TMP_AFFLU_NIVEAU (numéro niveau, libellé niveau, affluence, jour date, mois date) la date (jour et mois) et la valeur de la plus grande affluence de chaque niveau.

Exemple

```
CALL sp_niveau_maxaffluence( );
SELECT * FROM TMP_AFFLU_NIVEAU
```

numéro	libellé	affluence	jour	mois
1	vert	375	29	10
2	rouge	393	28	4
3	jaune	380	9	12